

Makefile 実践入門 — 中級者のための GNU Make 4.4

- 対象読者: 中級者（コマンドラインには慣れており、Makefileを一度は書いたことがある方）
- レベル: 中級
- 想定学習時間: 10～15時間
- 基準バージョン: GNU Make 4.4.1（2023年2月26日リリース。2026年7月時点の最新安定版）

目次

- はじめに
- 本書の使い方
- 学習環境
- 序章 なぜ今、Makefileなのか
- この章で学ぶこと
- Makeの歴史
- ビルドツールの現在地
- Makeが今も使われる理由
- 本書で学ぶこと
- 本章のまとめ
- 練習問題
- 第1章 Makefileの基本構造
- この章で学ぶこと
- 1.1 ルールの基本形
- 1.2 タブ文字の呪縛
- 1.3 デフォルトゴール
- 1.4 .PHONYとターゲットの使い分け
- 1.5 レシピの実行規則
- 1.6 コメントと行継続
- 本章のまとめ
- 練習問題
- 第2章 変数の理解がすべて
- この章で学ぶこと
- 2.1 変数代入の基本
- 2.2 代入演算子の種類（最重要）
- 再帰展開 = の罠
- 即時展開 := の利点
- 条件付き代入 ?=
- シェル代入 !=
- 2.3 自動変数

- 2.4 変数のフレーバー
- コマンドラインからの上書き
- 本章のまとめ
- 練習問題
- 第3章 ルールの高度な使い方
- この章で学ぶこと
- 3.1 パターンルール
- 3.2 静的パターンルール
- 3.3 グループ化ターゲット (&:)
- 3.4 二次展開 (.SECONDEXPANSION)
- 3.5 暗黙ルールとその無効化
- 3.6 中間ファイルと .NOTINTERMEDIATE
- 本章のまとめ
- 練習問題
- 第4章 関数とマクロ
- この章で学ぶこと
- 4.1 関数の基本形
- 4.2 ファイル操作関数
- \$(wildcard ...)
- \$(shell ...)
- \$(foreach var, list, text)
- 4.3 マクロ関数 \$(call)
- 4.4 動的生成 \$(eval)
- 4.5 4.4の新関数 \$(let) と \$(intcmp)
- \$(let var [var ...], list, text)
- \$(intcmp lhs, rhs [, lt-part [, eq-part [, gt-part]]])
- 4.6 \$(file) によるファイル読み書き
- 4.7 文字列関数 (頻出)
- 本章のまとめ
- 練習問題
- 第5章 並列ビルドとデバッグ
- この章で学ぶこと
- 5.1 並列実行 -j
- 5.2 出力の同期 --output-sync
- 5.3 ジョブサーバー
- 5.4 実行順序の制御 .WAIT と .NOTPARALLEL
- .WAIT (4.4の新機能)
- .NOTPARALLEL
- 5.5 --shuffle による依存漏れの検出 (4.4の目玉機能)
- 5.6 デバッグ
- make --debug

- `make -n` (ドライラン)
- `make -B` (強制再ビルド)
- `make -W file` (想定変更)
- 本章のまとめ
- 練習問題
- 第6章 依存関係自動生成と実践パターン
- この章で学ぶこと
- 6.1 依存関係の手動管理の限界
- 6.2 GCCによる依存自動生成
- 6.3 .dファイルをMakefileに読み込む
- 6.4 GNU Coding Standards の標準ターゲット
- 6.5 実践的なMakefileの全体像
- 6.6 条件分岐 (makeのディレクティブ)
- 本章のまとめ
- 練習問題
- 第7章 プラットフォーム対応と落とし穴
- この章で学ぶこと
- 7.1 プラットフォームごとのmake
- Linux
- macOS
- Windows
- Windowsでシェル非依存なレシピを書く
- 7.2 パスの問題
- 7.3 再帰makeと `$(MAKE)`
- 7.4 バージョンごとの後方非互換
- 7.5 POSIX.1-2024 とmakeの標準化
- 7.6 落とし穴の総まとめ
- 本章のまとめ
- 練習問題
- 付録A 練習問題の解答
- 練習問題 0.1
- 練習問題 0.2
- 練習問題 0.3
- 練習問題 1.1
- 練習問題 1.2
- 練習問題 1.3
- 練習問題 1.4
- 練習問題 2.1
- 練習問題 2.2
- 練習問題 2.3
- 練習問題 2.4

- 練習問題 2.5
- 練習問題 3.1
- 練習問題 3.2
- 練習問題 3.3
- 練習問題 3.4
- 練習問題 4.1
- 練習問題 4.2
- 練習問題 4.3
- 練習問題 4.4
- 練習問題 5.1
- 練習問題 5.2
- 練習問題 5.3
- 練習問題 5.4
- 練習問題 5.5
- 練習問題 6.1
- 練習問題 6.2
- 練習問題 6.3
- 練習問題 6.4
- 練習問題 6.5
- 練習問題 7.1
- 練習問題 7.2
- 練習問題 7.3
- 練習問題 7.4
- 練習問題 7.5
- 付録B 用語集
- 付録C 出典一覧
- 画像・図表の出典
- 免責

はじめに

本書は、Makefileを「一度は書いたことがある」中級者のために、GNU Makeを体系的かつ実践的に使いこなすための教科書です。

本書の使い方

- 各章は「学習目標 → 本文 → 本章のまとめ → 練習問題」の構成です。
- 本文中のコードは、可能な限りそのままコピーして実行できるようにしています。
- 各章末の練習問題の解答は付録Aに解説付きでまとめています。
- 本書はGNU Make 4.4.1を基準にしています。バージョン確認は `make --version` で行えます。4.4未満では本書の一部の機能（`--shuffle`、`.WAIT`、`$(let)`、`$(intcmp)` など）が使えない点に注意してください。

学習環境

- 本書の例は **Linux / macOS**（Homebrewで導入した最新の**make**）を想定しています。Windows の場合は WSL か MSYS2/MinGW を使うことで、ほぼ同じ例を実行できます（詳細は第7章）。
- 前提知識: シェルでの基本操作（cd、ls、ファイル編集）、C言語または他のコンパイル言語での小さなプログラムを書いた経験。

序章 なぜ今、Makefileなのか

この章で学ぶこと

- Makeが1970年代に誕生してから今日まで使われ続けている理由
- CMake / Ninja / Bazel / Task / Just など現代のビルドツールとの関係
- 自分がプロジェクトに導入するツールを選ぶ判断軸

Makeの歴史

Makeは、**Stuart Feldman** が1976年にベル研究所で開発したビルド自動化ツールです（1977年説も広く流通していますが、出典付きの記録では1976年4月完成とされています）。当時はC言語のソースコードをコンパイルして実行ファイルを作る手順を、タイムスタンプに基づいて自動化するものでした。

Makeの核となるアイデアは、実にシンプルです。

- 成果物（ターゲット）と、それが依存する材料（前提条件）を宣言する
- 前提条件がターゲットより新しければ、ターゲットを作るためのコマンド（レシピ）を実行する
- 変更されていないファイルは再ビルドしない

この「差分だけを再ビルドする」考え方は、現在のビルドシステムの基礎であり、40年以上経った今も変わりません。

GNU Makeは、Richard Stallmanが1988年頃に開発したGNUプロジェクト版のmakeで、現在の事実上の標準です。最新の安定版は 4.4.1（2023年2月26日リリース）で、2026年7月時点でもこれが最新です。

ビルドツールの現在地

2026年現在、Makeの周りには多くのビルドツールが存在します。大きく2つの系統に分けられます。

ツール	種類	特徴	主な用途
GNU Make	ビルドツール	タイムスタンプで差分ビルド。POSIX標準	小～中規模、汎用
NMAKE	ビルドツール	Microsoft製。Visual Studioに同梱	Windows/C++
CMake	ビルドシステム生成	MakefileやNinja ファイルを生成する	C/C++の標準

ツール	種類	特徴	主な用途
Ninja	ビルドツール	速度重視の低レベルエンジン	Chrome、Android、LLVM
Bazel	ビルドシステム	大規模モノレポ・サンドボックス・キャッシュ	大規模リポジトリ
Task / Just	タスクランナー	ビルド判定ではなく「タスク実行」に特化	プロジェクトの作業自動化

重要なのは、これらは競合というより役割が異なることです。

- CMakeはMakefileを直接生成する（CMakeの「ジェネレータ」としてMakeが使われる）。
- Ninjaは公式に「入力上位ビルドシステムが生成することを前提」と宣言しています。上位にCMakeやBazel、下位のエンジンとしてNinjaという構図です。Makefileは、こうした上位ツールの生成物としても今も生き続けています。
- Task / Justはタイムスタンプでビルドするmakeとは違い、プロジェクトの作業手順（テスト、デプロイなど）を登録して呼び出すタスクランナーです。

Makeが今も使われる理由

1. **ゼロインストール:** Linux / macOS / BSD にはデフォルトで同梱されており、POSIX規格の一部でもあります。
2. **汎用性:** ビルドだけでなく、データ処理のパイプラインやドキュメント生成など、ファイル間の依存関係がある処理なら何にでも使えます。
3. **エコシステムの前提:** Autotools や GNU Coding Standards をはじめ、多くのソフトウェアがMakefileを配布の標準としています。

一方で、弱点も理解しておきましょう。

- **タブ文字への依存**（第1章で詳述）
- **シェルへの依存:** レシピはシェルコマンドのため、rm -rf などプラットフォーム差異の影響を受ける（第7章）
- **依存関係の追跡が手動:** コンパイルの依存を自分で書く必要がある（第6章で自動化する方法を学びます）
- **並列ビルドの非決定性:** 前提条件の宣言漏れが -j を付けたときだけ現れる（第5章で --shuffle による対策を学びます）

本書で学ぶこと

- 第1～2章: Makefileの基本構造と変数の仕組み（土台）
- 第3～4章: ルールと関数を使った高度な記述
- 第5章: 並列ビルドとデバッグ
- 第6章: 依存関係の自動生成と実践的なMakefileの形
- 第7章: プラットフォーム対応と、よくある落とし穴の総まとめ

本章のまとめ

- Makeは1976年発明の、タイムスタンプによる差分ビルドを行うツール（1977年説も流通）
- GNU Make 4.4.1 が最新安定版（2023年2月26日、2026年7月現在も最新）
- CMake/Ninja/BazelはMakeの「上位」または「下位」のレイヤーであり、Makefileは生成物としても生き残っている
- Task/Justはビルドではなくタスク実行に特化した別種のツール

練習問題

練習問題 0.1 Makeの核となるアイデアを、ターゲット・前提条件・レシピの3語を使って説明してください。

練習問題 0.2 次のツールを「ビルドツール」「ビルドシステム生成」「タスクランナー」「低レベルビルドエンジン」「ビルドシステム（大規模向け）」の5つに分類してください: GNU Make / CMake / Ninja / Task / Bazel / Just

練習問題 0.3 CMakeがMakefileを生成できることからわかる、MakeとCMakeの関係を説明してください。

第1章 Makefileの基本構造

この章で学ぶこと

- ルールの3要素（ターゲット・前提条件・レシピ）と構文
- タブ文字の呪縛と、その回避策（.RECIPEPREFIX）
- デフォルトゴールと .PHONY の役割

1.1 ルールの基本形

Makefileの最小構成は、次の「ルール」です。

```
target: prerequisites
    recipe
```

- ターゲット（target）：作りたい成果物のファイル名
- 前提条件（prerequisites）：ターゲットより古くなったら再ビルドする対象のファイル
- レシピ（recipe）：ターゲットを作るために実行するコマンド

最初の例として、hello.c を gcc でコンパイルして hello という実行ファイルを作るMakefileを考えます。

```
hello: hello.c
    gcc -o hello hello.c
```

make hello と実行すると、makeは次の判断をします。

1. ターゲット hello が存在しない → レシピを実行
2. hello.c が hello より新しい → レシピを実行
3. どちらも無い → 「'hello' は最新です」と表示して何もしない

1.2 タブ文字の呪縛

レシピの行は、必ずタブ文字（TAB）で始めなければなりません。スペースで始めると、makeはエラーを出します。

```
Makefile:2: *** 分離した指示子がありません。 中止。
```

これはmakeの最も有名な落とし穴です。エディタがタブをスペースに変換する設定になっていると、レシピが突然動かなくなります。

回避策は3つあります。

1. エディタの設定で、Makefileではタブをスペースに展開しないようにする
2. エディタでタブ幅を見える化する
3. `$(.RECIPEPREFIX)` でタブ以外の文字を使う（GNU make 3.82以降）

`.RECIPEPREFIX` を使うと、レシピの先頭文字を変更できます。

```
.RECIPEPREFIX = >
hello: hello.c
> gcc -o hello hello.c
```

1.3 デフォルトゴール

引数なしで make を実行すると、makeは最初のターゲットをゴールとして扱います。これをデフォルトゴールと呼びます。

```
all: hello

hello: hello.c
    gcc -o hello hello.c
```

この例では all がデフォルトゴールです。一般的な慣習として、最初に all というターゲットを置き、実際に作りたいものをその前提条件に列挙します。

デフォルトゴールを明示的に指定するには `.DEFAULT_GOAL` を使います。

```
.DEFAULT_GOAL := all
```


ただし、`.d` ファイル（第6章）を `include` する場合、`include` した行がMakefileの先頭に展開されるため、意図しないターゲットがデフォルトゴールになる罠があります。これを防ぐには `.DEFAULT_GOAL` で明示するのが安全です。

1.4 .PHONYとターゲットの使い分け

`clean` や `test` のように、ファイルを作らない（あるいは削除する）ターゲットは、同じ名前のファイルが存在すると正しく動きません。

```
clean:
    rm -f *.o hello
```

もし `clean` という名前のファイルを誤って作ってしまうと、`make` は「`clean` は最新です」としてレシピを実行しません。

これを防ぐのが **.PHONY**（疑似ターゲット）です。

```
.PHONY: clean
clean:
    rm -f *.o hello
```

`.PHONY` に列挙されたターゲットは、常に「古い」とみなされ、必ずレシピが実行されます。`all`、`clean`、`install`、`test` などの実ファイルと対応しないターゲットは、すべて `.PHONY` に登録するのが基本です。

1.5 レシピの実行規則

レシピは1行ごとに別のシェルで実行されます。そのため、次の書き方は思った通りに動きません。

```
do:
    cd /tmp
    pwd    # 実行されるディレクトリは /tmp ではない！
```

各レシピ行は独立したシェルプロセスで実行されるため、`cd` の効果は次の行には引き継がれません。行を `\` で連結するか、`.ONESHELL` を使うと解決できます。

```
.ONESHELL:
do:
    cd /tmp
    pwd    # 今度は1つのシェルで実行されるので /tmp が表示される
```

また、`make` 変数をシェルに渡すには `$(VAR)`、シェル変数を参照するには `$$VAR` と書く必要があります。

```
print:
    @for f in $(ls *.c); do echo "$$f"; done
```

行頭の@は「このコマンド自体を表示しない」という意味です。デフォルトではmakeは実行するコマンドを標準出力に表示します。

1.6 コメントと行継続

- #以降はコメントです。
- 行末の\で次の行に継続できます。

```
# コメント
hello: hello.c \
    helper.c
    gcc -o hello hello.c helper.c
```

本章のまとめ

- ルール = ターゲット : 前提条件 + TABで始まるレシピ
- レシピは必ずTAB始まり（.RECIPEPREFIX で変更可能）
- 引数なしの make は最初のターゲットをデフォルトゴールにする
- .PHONY で実ファイルと対応しないターゲットを宣言する
- レシピは1行ごとに別シェルで実行される（.ONESHELL で1シェル化）

練習問題

練習問題 1.1 次のMakefileを make したとき、レシピが実行される条件を「ターゲットの有無」「前提条件の新旧」の観点から説明してください。

```
all: hello

hello: hello.c
    gcc -o hello hello.c
```

練習問題 1.2 タブ文字ではなくスペースでレシピを始めたとき、makeはどのような挙動をしますか。

練習問題 1.3 clean という名前の実ファイルが存在する場合、.PHONY で宣言されていない clean ターゲットはどうなりますか。

練習問題 1.4 次のMakefileで、do ターゲットのレシピを実行したときの出力を答えてください。

```
do:
    cd /tmp
    pwd
```

第2章 変数の理解がすべて

この章で学ぶこと

- 6種類の代入演算子（`=` `:=` `::=` `?=` `+=` `!=`）とその使い分け
- 再帰展開（`=`）と即時展開（`:=`）の違い
- 自動変数（`$@` `$<` `$^` `$?` `$*` `$|`）の使い方

2.1 変数代入の基本

Makefileの変数は、C言語のマクロに似ています。参照は `$(名前)` または `${名前}` で行います。単一文字なら `$名前` とも書けますが、習慣として常に `$(名前)` と書くのが安全です。

```
CC = gcc
CFLAGS = -Wall -O2

hello: hello.c
    $(CC) $(CFLAGS) -o hello hello.c
```

2.2 代入演算子の種類（最重要）

GNU makeには6種類の代入演算子があり、それぞれ展開のタイミングが異なります。これは中級者が最初に壁に当たるポイントです。

演算子	名前	展開のタイミング	意味
<code>=</code>	再帰展開	使用時に毎回展開	遅延評価（Lazy）
<code>:=</code>	即時展開	代入時に展開	早期評価（Eager）
<code>::=</code>	即時展開（POSIX）	代入時に展開	<code>:=</code> と同じ
<code>?=</code>	条件付き代入	—	未定義のときだけ代入
<code>+=</code>	追記	変数のフレーバー次第	既存値に追記
<code>!=</code>	シェル代入	代入時に実行	シェルコマンドの結果を代入

再帰展開 `=` の罠

`=` は、代入した文字列をそのまま記憶し、参照されたときに展開します。

```
A = foo
B = $(A) bar
A = baz

all:
    @echo $(B)    # 出力: baz bar
```

B を代入した時点では A はまだ foo でしたが、\$(B) を参照する時点では A は baz に変わっているため、baz bar と出力されます。これが「再帰展開」です。

さらに、= は自己参照の罠があります。

```
A = $(A) extra    # 無限ループ！
```

これを避けたい場合は += を使います（ただし空の変数への += は、GNU make 4.3以降で先頭スペースが付かなくなりました）。

即時展開 := の利点

:= は、代入した瞬間に右辺を展開します。

```
A = foo
B := $(A) bar    # この瞬間に A=foo で展開され、B は "foo bar" になる
A = baz

all:
    @echo $(B)    # 出力: foo bar
```

即時展開の利点は、実行時の値が安定していることです。大量の再帰展開はパフォーマンスにも悪影響を与えるため、慣習として変数は基本的に := で定義するのが実践的な流儀です。

::= はPOSIXで即時展開として標準化された演算子（POSIX 2012で追加。POSIX.1-2024でも踏襲）で、:= と同じ動作です。

条件付き代入 ?=

?= は、変数が未定義の場合にだけ代入します。既に定義されていれば何もしません。コマンドラインからの上書きと組み合わせて使います。

```
CFLAGS ?= -O2
```

make CFLAGS=-O0 と実行すると、コマンドラインで CFLAGS が定義済みになるため、?= は何もせず、-O0 が使われます。

シェル代入 !=

`!=` は右辺をシェルで実行し、結果を変数に代入します（GNU make 4.0で導入、BSD make互換）。`$(shell ...)` の代替です。

```
DATE := $(shell date +%Y%m%d)
# または
DATE != date +%Y%m%d
```

2.3 自動変数

自動変数は、ルール内で「今処理しているターゲット」や「前提条件」を参照するための特別な変数です。レシピ内と前提条件のリストで使えます（レシピ内で使うのが主）。

変数	意味
<code>\$@</code>	現在のターゲット名
<code>\$<</code>	最初の前提条件
<code>\$^</code>	すべての前提条件（重複は除く、順序はそのまま）
<code>\$+</code>	すべての前提条件（重複を含む）
<code>\$?</code>	ターゲットより新しい前提条件
<code>\$*</code>	パターンルールで一致した語幹（stem）
<code>\$ </code>	order-only前提条件のリスト（タイムスタンプは判定されない）
<code>\$(@D) \$(@F)</code>	<code>\$@</code> のディレクトリ部分 / ファイル名部分

例を見ましょう。

```
hello.o: hello.c
    gcc -c $< -o $@
```

これは次のように展開されます。

```
hello.o: hello.c
    gcc -c hello.c -o hello.o
```

`$^` の例です。

```
prog: main.o util.o
    gcc $^ -o $@
# 実際には: gcc main.o util.o -o prog
```

自動変数はレシピ内と前提条件のリストの両方で使えます。ただし前提条件のリストでは、通常は（二次展開なしでは）自動変数が展開されないため、`$@` などの利用には第3章で学ぶ二次展開（`.SECONDEXPANSION`）との組み合わせが必要です。実務上は、自動変数は主にレシピ内で使います。

2.4 変数のフレーバー

= と := の違いは「フレーバー（風味）」と呼ばれます。

- = の変数は再帰展開（recursively expanded）
- := / ::= の変数は即時展開（simply expanded）

+= は、対象の変数が既に即時展開なら即時展開として、再帰展開なら再帰展開として追記します。+= の左辺の変数が未定義なら、再帰展開として扱われます。

コマンドラインからの上書き

変数はコマンドライン引数で上書きできます。

```
$ make CFLAGS="-O0 -g"
```

このコマンドラインからの定義は、Makefile内の代入（= と :=）を上書きします。ただし `override` ディレクティブを使うと、Makefile側で強制的に上書きできます。

```
override CFLAGS += -fPIC
```

本章のまとめ

- 代入演算子は6種類あり、展開タイミングの理解が最重要
- = は参照時に展開（遅延）、:= は代入時に展開（即時）
- ?= は未定義時のみ、!= はシェル実行結果を代入（4.0+）
- 自動変数 \$@ \$< \$^ \$? でレシピを簡潔に書ける
- 変数はコマンドラインで上書きでき、`override` で防げる

練習問題

練習問題 2.1 次のMakefileで、`make all` を実行したときの出力を教えてください。

```
A = first
B = $(A) second
A = third

all:
    @echo $(B)
```

練習問題 2.2 同じく、`B := $(A) second` に変えた場合はどうなりますか。

練習問題 2.3 `CFLAGS ?= -O2` と書き、コマンドラインで `make CFLAGS=-O0` と実行したとき、`CFLAGS` の値はどうなりますか。

練習問題 2.4 次のレシピを自動変数 `$<` と `$@` を使って書き換えてください。

```
util.o: util.c
    gcc -c util.c -o util.o
```

練習問題 2.5 != 演算子の動作と、導入されたGNU makeのバージョンを説明してください。

第3章 ルールの高度な使い方

この章で学ぶこと

- パターンルールと静的パターンルールの違い
- グループ化ターゲット (`&:`) と、4.5での将来変更予告
- 二次展開 (`.SECONDEXPANSION`) で自動変数を前提条件に使う
- 暗黙ルールの仕組みと無効化

3.1 パターンルール

複数のファイルに同じレシピを適用したいときは、パターンルールを使います。`%`が「任意の文字列」に一致します。

```
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

これで `foo.c` から `foo.o` を、`bar.c` から `bar.o` を作るルールが1行で書けます。一致した `%` の部分は `$*` (システム) で参照できます。

```
%.o: %.c
    @echo "building $*"
    $(CC) $(CFLAGS) -c $< -o $@
```

3.2 静的パターンルール

静的パターンルールは、特定のターゲットにだけパターンを適用します。

```
$(OBJJS): %.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

`$(OBJJS)` に列挙されたファイルだけが、`%.o → %.c` のルールの対象になります。`$(OBJJS)` を列挙しておけば、ソースツリーに無関係な `.c` ファイルがあっても影響を受けません。

3.3 グループ化ターゲット (&:)

通常、1つのレシピは1つのターゲットを作ります。複数のファイルを1回のコマンドで生成する（例: パーサーが .c と .h を同時に生成する）場合は、**グループ化ターゲット**（GNU make 4.3+）を使います。

```
parser.c parser.h &: parser.y
    bison -d parser.y
```

&: で結ばれたターゲットはすべて同時に古いとみなされます。どちらか一方が更新されただけで、両方を作り直します。

注意（将来の変更予告）：GNU makeは将来、**パターンルールの複数ターゲット**（%.o %.h: %.y のように書いた場合）を、このグループ化ターゲットと同じ挙動に変更する予定です（4.5以降）。現在のバージョンでは複数ターゲットのパターンルールは、レシピが各ターゲットを個別に作ることを前提にしています。

3.4 二次展開 (.SECONDEXPANSION)

通常、前提条件のリストは**即時展開**されます。そのため、前提条件で自動変数（\$@ など）を使うことはできません。

```
obj/%.o: obj/%.c    # ここでは自動変数は使えない
```

しかし **.SECONDEXPANSION** を宣言すると、前提条件のリストを二段階に**展開**でき、自動変数が使えるようになります。

```
.SECONDEXPANSION:

obj/%.o: $$$(dirname $$@)/%.c
    $(CC) -c $$< -o $$@
```

ポイントは **\$\$** と2重に書くことです。1回目の展開で **\$** が残り、2回目の展開のタイミングで自動変数が解決されます。このテクニックは、**\$(eval)** と組み合わせてマクロを作るときに特に重要です（第4章）。

3.5 暗黙ルールとその無効化

GNU makeには、%.o: %.c や %.o: %.cpp などの**組み込みルール**（暗黙ルール）があらかじめ定義されています。

```
# この1行だけで、foo.o が作れる！
foo.o:
    # レシピなし → 暗黙ルール %.o: %.c が使われる
```


暗黙ルールは便利な一方で、意図しないターゲットに適用されてしまう落とし穴があります。例えば `clean` という名前の `.c` ファイルを作ってしまうと、`clean` ターゲットに暗黙ルールが適用されるかもしれません。

暗黙ルールを無効化するには次の方法があります。

```
# すべての組み込みルールを無効化
$(MAKEFLAGS) += -r
# または Makefile内で
.SUFFIXES:
```

`.SUFFIXES:` は接尾辞ルール（古い形式の暗黙ルール）を消します。 `-r` は組み込みルール全体を無効化します。

GNU make 4.4 では、暗黙ルールの探索アルゴリズムが改善され、予期せぬ暗黙ルールの連鎖（ought to exist）が起これにくくなっています。しかし、自分が制御できないMakefileを扱うときは、明示的なルールを書くのが基本です。

3.6 中間ファイルと `.NOTINTERMEDIATE`

パターンルールで作られたファイルは、最終ゴールの構築に使われた後、**中間ファイル**として自動的に削除されることがあります。

```
# main.o が中間ファイルとして扱われる例
main: main.o
    $(CC) $^ -o $@
```

中間ファイルの扱いを制御するには、次の特殊ターゲットを使います。

- `.INTERMEDIATE`: 明示的に中間ファイル扱いにする
- `.NOTINTERMEDIATE`: 中間ファイル扱いを無効化（4.4の新機能）
- `.SECONDARY`: 削除を防ぐ

```
.NOTINTERMEDIATE: obj/*.o
```

本章のまとめ

- パターンルール `%.o: %.c` で同種の変換を1本のルールにまとめる
- 静的パターンルールで対象を制限
- グループ化ターゲット `&`: で同時生成を宣言（パターンルールの複数ターゲットは将来変更予定）
- `.SECONDEXPANSION` で前提条件に自動変数を利用可能に
- 暗黙ルールは便利だが、`-r` / `.SUFFIXES:` で無効化できる

練習問題

練習問題 3.1 パターンルールと静的パターンルールの違いを説明してください。

練習問題 3.2 グループ化ターゲット `&`: の意味と、GNU makeのバージョンについて説明してください。

練習問題 3.3 暗黙ルールを無効化する2つの方法を答えてください。

練習問題 3.4 `.SECONDEXPANSION` を使うとき、前提条件で自動変数を `$$` と2重に書く理由を説明してください。

第4章 関数とマクロ

この章で学ぶこと

- ファイル操作関数 `$(wildcard)` `$(foreach)` `$(shell)`
- マクロ関数 `$(call)` `$(eval)` と、4.4の新関数 `$(let)` `$(intcmp)`
- `$(file)` によるファイル読み書き

4.1 関数の基本形

関数は `$(関数名 引数...)` の形で呼び出します。関数名と最初の引数の間はスペースで区切り、複数の引数はカンマで区切ります。

```
$(wildcard src/*.c)
```

4.2 ファイル操作関数

`$(wildcard ...)`

指定したワイルドカードに一致する実在するファイルのリストを返します。

```
SRCS := $(wildcard src/*.c)
```

GNU make 4.3 以降、`$(wildcard)` は結果をソートした順で返すことが保証されています。それ以前は環境やファイルシステムによって順序が不定でした（実質的にはソートされていましたが、保証はありませんでした）。

`$(shell ...)`

シェルコマンドを実行し、その標準出力を返します。改行はスペースに変換されます。

```
GIT_HASH := $(shell git rev-parse --short HEAD)
```

落とし穴として、\$(shell) はレシピの外でも実行されるため、ビルドを走らせた回数だけ外部コマンドが実行されます。コストの高いコマンドは ?= や条件付きで使うとよいでしょう。

\$(foreach var, list, text)

list の各要素を var に代入しながら text を展開し、結果をスペース区切りで連結します。

```
DIRS = src include lib
OBJDIRS := $(foreach d, $(DIRS), obj/$(d))
# 結果: obj/src obj/include obj/lib
```

4.3 マクロ関数 \$(call)

\$(call name, args...) は、定義済みの変数（マクロ）を関数のように呼び出します。引数は \$(1)、\$(2)、... で参照します。

```
define compile
    $(CC) $(CFLAGS) -c $(1) -o $(2)
endef

util.o: util.c
    $(call compile,util.c,util.o)
```

define ... endef で複数行の変数（マクロ）を定義できます。

4.4 動的生成 \$(eval)

\$(eval ...) は、文字列をMakefileの一部として評価します。マクロと組み合わせると、繰り返しを展開してルールを動的生成できます。

```
define make_rule
$(1).o: $(1).c
    $$$(CC) $$$(CFLAGS) -c $$< -o $$@
endef

$(eval $(call make_rule,util))
$(eval $(call make_rule,main))
```

\$(eval) の中では、展開のタイミングに注意が必要です。define 内の \$\$ は1回目の展開（call時）で \$ に戻り、\$(eval) が2回目の展開を行うことで、\$\$@ が正しく自動変数になります。

この「\$(eval) の展開タイミング」は、中級者が陥る最大級の難所です。落ち着いて、何が何回目の展開で解決されるかを追ってください。

4.5 4.4の新関数 \$(let) と \$(intcmp)

\$(let var [var ...], list, text)

\$(let) は、**text** の中だけで有効なローカル変数を作る関数です（GNU make 4.4で導入）。**list** の値を各 **var** に割り当て、**text** を展開します。

```
define twice
    $(let x, $(1), echo $$x) $(x))
endef

all:
    $(call twice,hello)
    # 出力: hello hello
```

var は複数指定できます。この場合、**list** の各語が順に各 **var** に割り当てられます。

```
$(let a b, 1 2, $(a)-$(b))
# 結果: 1-2
```

\$(intcmp lhs, rhs [, lt-part [, eq-part [, gt-part]]])

\$(intcmp) は数値比較を行う関数です（4.4で導入）。引数は次のとおりです。

- **lt-part**: **lhs < rhs** のとき返す値
- **eq-part**: **lhs = rhs** のとき返す値
- **gt-part**: **lhs > rhs** のとき返す値（省略時は **eq-part** を流用）

```
VERSION := $(intcmp 4,4,small,equal,large)
# 結果: equal

VERSION := $(intcmp 4,4,yes,no)
# 結果: no （第4引数が「等しいとき」の値）
```

2引数だけの **\$(intcmp 4,4)** は、等しいとき **1**、等しくないとき **0** を返します。

4.6 \$(file) によるファイル読み書き

\$(file op FILENAME[, text]) でファイルへの書き込み・読み込みを行います（書き込み機能は4.0、読み込み **\$(file < FILE)** は4.2で導入）。

```
$(file > output.txt,hello world)
CONTENT := $(file < output.txt)
```

> は上書き、**>>** は追記です。

4.7 文字列関数（頻出）

関数	意味
\$(subst from,to,text)	文字列置換
\$(patsubst pattern,repl,text)	パターン置換（%を使える）
\$(filter pattern...,text)	条件に一致する語を抽出
\$(filter-out pattern...,text)	条件に一致しない語を抽出
\$(sort list)	ソート（重複除去）
\$(word n,text)	n番目の語
\$(firstword text)	最初の語

\$(patsubst) は、変数参照の置換構文 \$(var:%.c=%.o) の一般形です。

```
OBJS := $(SRCS:%.c=%.o)
# は次と同じ
OBJS := $(patsubst %.c,%.o,$(SRCS))
```

本章のまとめ

- \$(wildcard) \$(shell) \$(foreach) でファイル・コマンド・リストを扱う
- \$(call) + define でマクロを定義し、\$(eval) でルールを動的生成
- 4.4の新関数 \$(let)（ローカル変数）と \$(intcmp)（数値比較）
- \$(file) は書き込み4.0、読み込み4.2で追加
- \$(patsubst) は \$(var:%.c=%.o) の一般形

練習問題

練習問題 4.1 \$(foreach) を使って、SRC_DIRS = src1 src2 src3 から build/src1 build/src2 build/src3 を生成する式を書いてください。

練習問題 4.2 \$(eval) を使うとき、define 内の自動変数を \$\$ と書く理由を説明してください。

練習問題 4.3 \$(wildcard *.c) の結果がGNU make 4.3以降でソートされることの実務上の意味を説明してください。

練習問題 4.4 \$(intcmp) と \$(let) が導入されたGNU makeのバージョンを答え、それぞれの用途を1文で説明してください。

第5章 並列ビルドとデバッグ

この章で学ぶこと

- `-j` による並列ビルドと `--output-sync` による出力整形
- ジョブサーバーの仕組み
- `.WAIT` / `.NOTPARALLEL` による実行順序の制御
- `--shuffle` による依存漏れの検出（4.4の目玉機能）

5.1 並列実行 `-j`

`make -j4` と指定すると、依存関係のないルールを最大4つ同時に実行します。`-j` なしの場合は直列実行です。

```
$ make -j$(nproc)
```

並列ビルドは劇的に速くなる一方、**前提条件の宣言漏れ**という厄介なバグを引き起こします。直列なら偶然順番どおりに実行されて成功するのに、並列では順序が変わって失敗するのです。

5.2 出力の同期 `--output-sync`

並列実行中、複数のレシピの出力が混ざって読みにくくなります。`--output-sync (-O)` で出力をまとめられます。

```
make -j4 --output-sync=target
```

モード	動作
none	同期しない
line	1行ずつ同期
target	ターゲットごとにまとめて出力
recurse	再帰makeまで含めて同期

5.3 ジョブサーバー

`-j` で並列ビルドするとき、`make`は子プロセス（サブ`make`を含む）と「ジョブサーバー」を通じて並列度を共有します。

- GNU make 4.4 以降、UNIX系ではデフォルトで名前付きパイプ（FIFO）方式を使います（`--jobserver-style=fifo`）。
- 一部の環境（Cygwin、GNU/Hurd）では互換性のためパイプ方式に戻されます。
- Windowsでは名前付きセマフォ方式（`--jobserver-style=sem`）になります。

レシピからサブmakeを呼ぶときは、必ず `make` ではなく `$(MAKE)` を使います。`$(MAKE)` を使うとジョブサーバーへの接続情報 (`MAKEFLAGS`) が継承され、`-j` の並列度が守られます (第7章で詳述)。

5.4 実行順序の制御 `.WAIT` と `.NOTPARALLEL`

`.WAIT` (4.4の新機能)

前提条件のリストに `.WAIT` を置くと、その左側の前提がすべて完了するまで右側を開始しないことを強制します。

```
all: gen_sources .WAIT compile
```

POSIX.1-2024 でも `.WAIT` が標準化されており、GNU make 4.4はその先行実装です。

`.NOTPARALLEL`

`.NOTPARALLEL` に列挙したターゲットのビルドを直列化します (4.4以降は前提条件の指定を受け付けます)。

```
.NOTPARALLEL: data_gen
```

5.5 `--shuffle` による依存漏れの検出 (4.4の目玉機能)

並列ビルド特有のバグ (前提条件の漏れ) を検出するための、GNU make 4.4 の新機能です。`--shuffle` は前提条件やゴールの順序を意図的に並べ替えて実行し、ビルドが壊れるかどうかを確認します。

```
make -j4 --shuffle
make -j4 --shuffle=random
make -j4 --shuffle=reverse
make -j4 --shuffle=seed=1234
```

- `random`: ランダムに並べ替え
- `reverse`: 逆順 (最も検出しやすい)
- `seed=番号`: シードを固定して再現可能にする

CIで `make -j4 --shuffle=reverse` を実行しておくと、宣言漏れの依存がビルド失敗として検出されます。

5.6 デバッグ

`make --debug`

フラグ	動作
<code>--debug=all</code>	すべての情報を表示

フラグ	動作
<code>--debug=print</code>	レシピの表示
<code>--debug=why</code>	なぜ再ビルドするのかを表示 (4.4)
<code>--trace</code>	レシピ表示のトレース出力 (4.0導入。4.4で <code>--debug=print,why</code> と等価に変更)

```
$ make --debug=why
GNU Make 4.4
Prerequisite 'hello.c' is newer than target 'hello'.
```

make -n (ドライラン)

コマンドを実行せず、何が実行されるかを表示します。`$(MAKE)` を使った再帰makeも、`-n` で正しく伝播します。

```
$ make -n
gcc -Wall -O2 -c hello.c -o hello.o
gcc hello.o -o hello
```

make -B (強制再ビルド)

すべてのターゲットを古いものとして扱い、すべて再ビルドします。

make -W file (想定変更)

`file` が変更されたものとして再ビルドします。依存関係の確認に便利です。

本章のまとめ

- `-jN` で並列ビルド、`--output-sync` で出力を整形
- ジョブサーバー (4.4以降FIFO方式) により並列度を共有。サブmakeは `$(MAKE)` を使う
- `.WAIT` (4.4) と `.NOTPARALLEL` で実行順序を制御
- `--shuffle` (4.4) は依存宣言漏れの検出に有効
- `--debug=why` / `--trace` で再ビルドの理由を調べる (`--debug=why`は4.4で追加)

練習問題

練習問題 5.1 `make -j4` を実行したときに、前提条件の宣言漏れが原因で失敗することがあるのはなぜですか。

練習問題 5.2 `--output-sync=target` の動作を説明してください。

練習問題 5.3 レシピ内からサブmakeを呼ぶときに `make` ではなく `$(MAKE)` を使うべき理由を説明してください。

練習問題 5.4 `--shuffle` は何を検出するための機能ですか。 `--shuffle=reverse` と `--shuffle=seed=1234` の違いも教えてください。

練習問題 5.5 GNU make 4.4で、ジョブサーバーのUNIX系デフォルト方式がどう変わったかを説明してください。

第6章 依存関係自動生成と実践パターン

この章で学ぶこと

- GCCの `-MMD` `-MP` による依存関係の自動生成
- `.d` ファイルを `include` で読み込む実践パターン
- GNU Coding Standards に沿った標準ターゲットの設計
- 実践的なMakefileの全体像

6.1 依存関係の手動管理の限界

これまでの例では、ヘッダファイルの依存を手動で書いていました。

```
main.o: main.c util.h common.h
    $(CC) $(CFLAGS) -c $< -o $@
```

しかしヘッダファイルの依存は、`#include` を追うたびに手で更新する必要があり、漏れやすいものです。ヘッダを変更しても再ビルドされないという、最悪のバグの温床になります。

6.2 GCCによる依存自動生成

GCC（およびclang）は、コンパイルの副作用として依存関係ファイル（`.d`）を生成できます。

```
gcc -MMD -MP -MF main.d -c main.c -o main.o
```

オプション	意味
<code>-MMD</code>	コンパイル時に <code>.d</code> ファイルを生成（システムヘッダは除外）
<code>-MP</code>	各依存ヘッダにダミーのphonyターゲットを追加（ヘッダ削除時のエラー回避）
<code>-MF file</code>	<code>.d</code> ファイルの出力先を指定

生成される `main.d` は次のような内容です。

```
main.o: main.c common.h util.h

common.h:
util.h:
```

6.3 .dファイルをMakefileに読み込む

GNU makeは、includeしたファイル自体が更新されると、Makefileを再生成するためのルールを探し、見つければ自動で再実行します。これを利用したのが以下のパターンです。

```
SRCS := $(wildcard src/*.c)
OBJS := $(SRCS:%.c=%.o)
DEPS := $(OBJS:.o=.d)

-include $(DEPS)

%.o: %.c
    $(CC) $(CFLAGS) -MMD -MP -MF $(@:.o=.d) -c $< -o $@
```

ポイント:

- \$(OBJS:.o=.d) で .o を .d に変換
- -include (ハイフン付き) なら、.d ファイルが存在しなくてもエラーにならない
- -MMD -MP をコンパイル時に指定するので、.d は自動的に最新に保たれる
- ヘッダが更新されると .d 経由で依存が再評価され、再ビルドが正しく走る

このパターンが、2026年現在のGNU Makeを使った依存管理の標準形です。GNU makeマニュアルの「Generating Prerequisites Automatically」(4.14節)でも、-M 系オプションと include の組み合わせが推奨されています。

⚠: .d ファイルを include で読み込む場合、その行が展開されるとMakefileの「先頭」に .d の内容 (つまり最初のターゲット) が来ることがあります。これでデフォルトゴールが変わってしまうを防ぐため、.DEFAULT_GOAL を先頭で明示しましょう (第1章)。

6.4 GNU Coding Standards の標準ターゲット

GNUプロジェクトのソフトウェアでは、Makefileに次のターゲットを実装するのが慣習です (GNU Coding Standards 「Makefile Conventions」)。

ターゲット	動作
all	デフォルトのビルド
clean	ビルド成果物を削除
install	システムへインストール
uninstall	アンインストール
check / test	テスト実行
distclean	clean + 設定ファイル類も削除

また、変数は標準の名前を使います。CC (Cコンパイラ)、CFLAGS、prefix (インストール先)、DESTDIR (ステージング用) などです。

```
CC = gcc
CFLAGS ?= -O2 -Wall
prefix ?= /usr/local

all: hello

install: hello
    install -m 755 hello $(DESTDIR)$(prefix)/bin/hello

check: hello
    ./hello --selftest

clean:
    rm -f *.o *.d hello

.PHONY: all install check clean
```

DESTDIR はパッケージングのとき、実際のルートではなく仮のディレクトリにインストールするための変数です。make DESTDIR=/tmp/stage install と実行します。

6.5 実践的なMakefileの全体像

ここまでの知識をまとめた、中規模CプロジェクトのMakefile例です。

```
CC      := gcc
CFLAGS  := -O2 -Wall -Wextra
LDFLAGS :=
LIBS     := -lm

SRCS     := $(wildcard src/*.c)
OBJS     := $(SRCS:%.c=%.o)
DEPS     := $(OBJS:.o=.d)
TARGET  := app

.DEFAULT_GOAL := all

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(LDFLAGS) $^ -o $@ $(LIBS)

%.o: %.c
    $(CC) $(CFLAGS) -MMD -MP -MF $(@:.o=.d) -c $< -o $@

-include $(DEPS)

clean:
    rm -f $(OBJS) $(DEPS) $(TARGET)

check: $(TARGET)
    ./$$(TARGET) --test

install: $(TARGET)
    install -m 755 $(TARGET) $(DESTDIR)$(prefix)/bin/$(TARGET)

.PHONY: all clean check install
```

特徴:

- 変数は `:=` で定義（展開タイミングを安定化）
- `wildcard` でソースを自動列挙
- `-MMD -MP` で依存を自動生成、`-include` で読み込み
- 標準ターゲット `all` / `clean` / `check` / `install` を `.PHONY` 登録
- `$(LDFLAGS)` と `$(LIBS)` を分離（リンク時フラグとライブラリ）

6.6 条件分岐（makeのディレクティブ）

`ifeq` などの条件ディレクティブで、環境に応じて動作を切り替えられます。

```
ifeq ($(OS),Windows_NT)
    RM = del /Q
else
    RM = rm -f
endif
```

その他のディレクティブ:

ディレクティブ	意味
ifdef NAME / ifndef NAME	変数の定義有無で分岐
ifeq (a,b) / ifneq (a,b)	値の一致で分岐
else / endif	分岐の終端
define	複数行マクロの定義（第4章）
override	コマンドライン上書きを防ぐ（第2章）

本章のまとめ

- GCC `-MMD -MP -MF` で依存関係を自動生成する
- `-include $(DEPS)` と `.DEFAULT_GOAL` の明示で、依存管理の標準形を実装
- GNU Coding Standards に沿った標準ターゲット（`all/clean/install/check`）と変数名
- `DESTDIR` でステージングインストールに対応
- `ifeq / ifdef` などの条件ディレクティブで環境分岐

練習問題

練習問題 6.1 `-MMD` と `-MP` の役割をそれぞれ説明してください。

練習問題 6.2 `-include`（ハイフン付き）と通常の `include` の違いを説明してください。

練習問題 6.3 `.d` ファイルを`include`する際、`.DEFAULT_GOAL` を明示すべき理由を説明してください。

練習問題 6.4 `make DESTDIR=/tmp/stage install` の `DESTDIR` の意味を説明してください。

練習問題 6.5 GNU Coding Standards で定められた標準ターゲットを4つ挙げ、それぞれの動作を説明してください。

第7章 プラットフォーム対応と落とし穴

この章で学ぶこと

- Linux / macOS / Windows での`make`の導入と差異
- レシピのシェル依存と対策

- 再帰makeと \$(MAKE) の正しい使い方
- バージョンごとの後方非互換と、POSIX.1-2024

7.1 プラットフォームごとのmake

Linux

GNU makeが標準でインストールされています。最新のディストリビューションは4.4系です。レシピのシェルは /bin/sh が使われ、.SHELLFLAGS の既定値は -c です（.POSIX 指定時は -ec）。

macOS

macOSに同梱されているmakeはGNU makeですが、バージョンが古いことがあります。Homebrewで最新版を gmake という名前で導入できます（システムのmakeを置き換えません）。

```
$ brew install make
$ gmake --version    # GNU Make 4.4.1
```

プロジェクトでは gmake を前提にするか、make のバージョンチェックを入れると安全です。

Windows

GNU makeはWindows標準に含まれません。主な選択肢は次の4つです。

環境	特徴
WSL	wsl --install でLinuxディストリを導入し、本物のLinux版GNU makeを実行。最もおすすめ
MSYS2 / MinGW	ネイティブWindows用のUnix風ビルド環境。pacmanでGNU makeを導入
Cygwin	Unix環境のエミュレーション。GNU make利用可
NMAKE	Visual Studio付属のMicrosoft製make。構文がGNU makeと異なる

Windowsでは、次の点に注意します。

- シェルの扱い: Unix版makeは SHELL 環境変数を見りますが、Windows（MS-DOS系）では makefile内の SHELL 変数が参照されます。SHELL=/bin/sh を置くと、PATHからUnix風シェルを探して使います。
- ジョブサーバー: Windowsでは名前付きセマフォ方式（--jobserver-style=sem）です。
- 一時ファイル: Windowsでは TMP / TEMP に作成されます（4.4では MAKE_TMPDIR で上書き可能）。

Windowsでシェル非依存なレシピを書く

`rm -rf`、`mkdir -p` は `cmd.exe` では動作しないため、そのままではWindows（コマンドプロンプト）で動きません。対策は次のいずれかです。

1. WSL / MSYS2 を利用して Unix シェルを使う（最も実用的）
2. レシピをシェル非依存にする

```
# シェルに依存しない例
clean:
    rm -f $(OBSJ)    # Unixシェル前提。WSL/MSYS2で動く
```

7.2 パスの問題

`make` は `/` を認識しますが、Windowsネイティブコマンドでは `\` がエスケープと衝突しやすい問題があります。MSYS2 / Cygwin 経由の `sh.exe` を使うことで、Unix風のパス記述がそのまま通ります。

7.3 再帰makeと `$(MAKE)`

複数ディレクトリのMakefileを管理するとき、トップのMakefileからサブディレクトリのmakeを呼びたくなります。

```
subdir:
    cd sub && $(MAKE) all
```

必ず `$(MAKE)` を使い、`make` と書いてはいけません。理由は次のとおりです。

- `MAKEFLAGS` (`-j` の並列度、`--output-sync` などの設定) が子プロセスへ伝播する
- ジョブサーバーとの接続が引き継がれ、`-j` の並列度が守られる
- `-n` (ドライラン) や `-s` などの挙動が正しく伝わる

```
# 誤: 並列度が守られない
subdir:
    cd sub && make all

# 正
subdir:
    cd sub && $(MAKE) all
```

7.4 バージョンごとの後方非互換

GNU makeのNEWSには、バージョンごとの「後方非互換の警告」がまとめられています。特に注意すべき点:

バージョン	変更点
4.4	ifeq のカンマ解析、グループ化ターゲットの再ビルド判定、MAKEFLAGS の更新タイミング、--print-directory の優先順位が変更
4.3	関数内の # の扱い、\$(wildcard) のソート、空変数への += が先頭スペースを付けない、-j の上限が 63→4095
4.0	!= と ::= が追加
3.81	二次展開 (.SECONDEXPANSION) が追加

古いMakefileを新しいmakeで動かすときは、これらの変更を確認しましょう。

7.5 POSIX.1-2024 とmakeの標準化

2024年に公開された POSIX.1-2024 (IEEE Std 1003.1-2024) では、make ユーティリティが大幅に拡張され、以下の機能が標準仕様に取り込まれました。

- .WAIT (前提条件の順序強制)
- != (シェル代入)
- ::= (即時展開、POSIX 2012で追加済み) と :::= (右辺を即時展開する遅延評価形式。GNU make 4.4.1では未実装で、開発版の機能)
- ?= (条件付き代入) と += (追記)

GNU make 4.4 の .WAIT は「今後のPOSIXで必要になる」との注記付きで先行実装されたもので、両者の統合が図られています。 .POSIX 特殊ターゲットを宣言すれば、POSIX準拠の挙動で動作します。

7.6 落とし穴の総まとめ

落とし穴	対策
タブ vs スペース	.RECIPEPREFIX か、エディタ設定でタブを維持
レシピが行ごとに別シェル	.ONESHELL か \ での連結
= の遅延展開で混乱	基本 := を使う
.PHONY 忘れ	実ファイルと対応しないターゲットは必ず宣言
暗黙ルールが適用される	-r / .SUFFIXES: で無効化
並列で依存漏れが発覚	--shuffle でCI時に検出
サブmakeを make と書く	\$(MAKE) を使う
ヘッダ変更が反映されない	-MMD -MP + -include で依存自動化
Windowsで rm -rf が動かない	WSL/MSYS2を使うかシェル非依存に

本章のまとめ

- Linuxは標準、macOSは `gmake` (Homebrew)、Windowsは WSL/MSYS2 を推奨
- Windowsではシェル・ジョブサーバー・一時ファイルの扱いが異なる
- 再帰makeは `$(MAKE)` を使い、ジョブサーバーとフラグを継承する
- バージョンごとの後方非互換に注意 (4.0/4.3/4.4)
- POSIX.1-2024で `.WAIT` や各代入演算子が標準化された

練習問題

練習問題 7.1 macOSで最新のGNU makeを `gmake` という名前で導入する方法と、その理由を説明してください。

練習問題 7.2 WindowsでGNU makeを使う主な3つの方法を挙げてください。

練習問題 7.3 再帰makeで `$(MAKE)` を使うべき理由を2つ挙げてください。

練習問題 7.4 GNU make 4.3 で `$(wildcard)` の挙動がどう変わったか、その影響を説明してください。

練習問題 7.5 POSIX.1-2024 で標準化されたmakeの機能を3つ挙げてください。

付録A 練習問題の解答

練習問題 0.1

解答: ターゲットとは作りたい成果物、前提条件はその材料、レシピはターゲットを作るコマンドです。Makeは前提条件がターゲットより新しい（またはターゲットが存在しない）とき、レシピを実行してターゲットを生成します。変更のないファイルは再ビルドされないため、差分だけを効率よくビルドできます。

練習問題 0.2

解答: ビルドツール = GNU Make / ビルドシステム生成 = CMake / 低レベルビルドエンジン = Ninja / タスクリランナー = Task, Just / ビルドシステム（大規模向け） = Bazel

練習問題 0.3

解答: CMakeはMakefileやNinjaファイルを「生成する」上位ツールです。CMake自身は低レベルなビルドを行わず、生成したMakefileが実際のビルドを担います。つまりCMakeとMakeは競合ではなく、MakefileはCMakeの生成物としても生き続けています。

練習問題 1.1

解答: `make` はデフォルトゴールの `all` をビルドします。 `all` の前提は `hello` です。 `hello` が存在しない、または `hello.c` が `hello` より新しい場合、レシピ `gcc -o hello hello.c` が実行されます。 `hello` が存在し `hello.c` より新しければ何も実行されません。

練習問題 1.2

解答: `make` は「分離した指示子がありません」などのエラーを出して中止します。 レシピ行は必ずタブ文字で始める必要があります。

練習問題 1.3

解答: `clean` という名前の実ファイルが存在し、`.PHONY` に登録されていない場合、`make` は `clean` を「最新」とみなしてレシピを実行しません。 実ファイルと対応しないターゲットは `.PHONY` に登録する必要があります。

練習問題 1.4

解答: 各レシピ行は別々のシェルで実行されるため、`cd /tmp` の効果は `pwd` に引き継がれません。 `pwd` は元のディレクトリを表示します (`.ONESHELL` を指定しない限り)。

練習問題 2.1

解答: `baz bar` です。 `=` (再帰展開) では、`B` は参照時 (`echo` 実行時) に `A` を展開するため、その時点の `A` の値 (`third`) が使われます。

練習問題 2.2

解答: `first bar` です。 `:=` (即時展開) では代入時に展開されるため、`A=first` の時点の値で `B` が確定します。

練習問題 2.3

解答: `-o0` のままです。 コマンドラインで定義された変数はMakefile内の `?=` では上書きされず、`?=` は「未定義のときだけ」代入します。

練習問題 2.4

解答: `gcc -c $< -o $@` です。 `$<` は最初の前提条件 (`util.c`)、`$@` はターゲット (`util.o`) に展開されます。

練習問題 2.5

解答: != は右辺をシェルで実行し、その標準出力を変数に代入します。GNU make 4.0 で導入されました。
\$(shell ...) の代替です。

練習問題 3.1

解答: パターンルール (%.o: %.c) は、パターンに一致するすべてのターゲットに適用されます。静的パターンルール (\$(OBJS): %.o: %.c) は、リストに列挙したターゲットにだけ適用されます。静的パターンルールは、ソースツリーに無関係なファイルが存在しても影響を受けない利点があります。

練習問題 3.2

解答: グループ化ターゲット &: で結ばれた複数のターゲットは、すべて同時に古いとみなされ、1回のレシピで生成されます。パーサーが .c と .h を同時生成するような場合に使います。GNU make 4.3 で導入されました。

練習問題 3.3

解答: (1) コマンドラインで -r を指定、または \$(MAKEFLAGS) += -r で組み込みルール全体を無効化。(2) .SUFFIXES: を空で宣言して接尾辞ルールを消す。の2つです。

練習問題 3.4

解答: .SECONDEXPANSION では前提条件のリストが2段階で展開されます。1回目の展開で \$\$ が \$ に戻り、2回目の展開の時点で自動変数が解決されるからです。単一の \$ だと1回目の展開で \$(dir \$@) が未定義になり空になってしまいます。

練習問題 4.1

解答: \$(foreach d, \$(SRC_DIRS), build/\$(d)) です。結果は build/src1 build/src2 build/src3 になります。

練習問題 4.2

解答: \$(eval) の展開は2段階あります。1回目 (\$(call) などによる展開) で \$\$ が \$ に戻り、2回目 (\$(eval) による評価) で自動変数が解決されます。1重の \$ だと1回目の展開で変数参照が空になってしまうため、2重に書く必要があります。

練習問題 4.3

解答: 4.3以降、\$(wildcard) はソート済みの結果を返すため、ファイルの読み取り順に依存した非決定的なビルドが減ります。以前はディレクトリエントリの順序に依存していたため、環境によって結果が異なることがありました。

練習問題 4.4

解答: どちらもGNU make 4.4 で導入されました。`$(intcmp)` は2つの数値を比較し、大小・一致に応じた値を返す関数、`$(let)` は式の中でだけ有効なローカル変数を定義する関数です。

練習問題 5.1

解答: 直列実行では、偶然ターゲットのビルドに必要な先行処理が実行順序の都合で済んでいることがあります。並列にすると順序が保証されず、宣言していない依存がまだ作られていない状態でレシピが実行され、失敗します。前提条件として正しく宣言することで、makeが並列でも順序を保証できます。

練習問題 5.2

解答: `--output-sync=target` は、各ターゲットのレシピの出力をターゲットごとにまとめて、他のターゲットの出力と混ざらないように同期して表示します。`-j` で並列実行中に出力が混ざるのを防ぎます。

練習問題 5.3

解答: `$(MAKE)` を使うと、`MAKEFLAGS` (`-j` の並列度や `--output-sync` など) とジョブサーバーへの接続が子プロセスに継承されます。`make` と直書きすると、並列度が守られず、`-n` (ドライラン) も正しく伝播しません。

練習問題 5.4

解答: `--shuffle` は、前提条件の宣言漏れ (依存関係の漏れ) を検出する機能です。`--shuffle=reverse` は前提条件を逆順で実行するため最も漏れを検出しやすく、`--shuffle=seed=1234` はシードを固定するため同じ並べ替えを再現できます。

練習問題 5.5

解答: GNU make 4.4 以降、UNIX系ではジョブサーバーのデフォルト方式が名前付きパイプ (FIFO) 方式 (`--jobserver-style=fifo`) に変わりました。一部の環境 (Cygwin・GNU/Hurd) では互換性のため従来のパイプ方式に戻されます。

練習問題 6.1

解答: `-MMD` は、コンパイル時に依存関係を `.d` ファイルへ自動生成します (システムヘッダは除外)。`-MP` は、各依存ヘッダに対してダミーのphonyターゲットを追加し、ヘッダが削除された場合のmakeのエラーを防ぎます。

練習問題 6.2

解答: `-include` (ハイフン付き) は、対象ファイルが存在しなくてもエラーにせず続行します。通常の `include` はファイルが見つからないとエラーになります。初回ビルド時は `.d` がまだ無いため、`-include` が

必須です。

練習問題 6.3

解答: `include` した `.d` ファイルの内容がMakefileの先頭付近に展開されると、そこに書かれた最初のターゲット（例: `main.o: ...`）がデフォルトゴールになってしまう可能性があるためです。`.DEFAULT_GOAL` で明示すれば、意図しないターゲットがデフォルトになるのを防げます。

練習問題 6.4

解答: `DESTDIR` は、インストール先のルートディレクトリを仮の場所にずらすための変数です。`install` は `$(DESTDIR)$(prefix)/bin/...` へコピーされます。パッケージングやクロスビルドのとき、実際のシステムに影響させずステージングディレクトリへ一時インストールするのに使います。

練習問題 6.5

解答: `all`（デフォルトのビルド）、`clean`（ビルド成果物の削除）、`install`（システムへのインストール）、`check`（テストの実行）。このほかに `uninstall`（アンインストール）、`distclean`（設定を含む完全削除）などがあります。

練習問題 7.1

解答: `brew install make` で導入できます。Homebrewのmake formulaは、システムのmakeを置き換えないよう `gmake` という名前で導入されます。macOS同梱のmakeはバージョンが古い可能性があるため、最新版を `gmake` として使うのが安全です。

練習問題 7.2

解答: (1) WSL（Linuxディストリを導入して本物のGNU makeを実行）、(2) MSYS2/MinGW（Unix風ネイティブビルド環境）、(3) Cygwin（Unix環境のエミュレーション）。Visual StudioのNMAKEもありますが、構文がGNU makeと異なります。

練習問題 7.3

解答: (1) `MAKEFLAGS`（`-j` の並列度、`--output-sync` など）が子プロセスに伝播する。(2) ジョブサーバーとの接続が引き継がれ、`-j` の並列度が守られる。加えて `-n`（ドライラン）などの挙動も正しく伝わります。

練習問題 7.4

解答: GNU make 4.3 以降、`$(wildcard)` は結果をソートした順で返すようになりました。以前はディレクトリの読み取り順に依存していたため、環境やファイルシステムによってビルド結果（オブジェクトのリンク順など）が変わる可能性がありました。ソートにより結果が決定的になりました。

練習問題 7.5

解答: (1) `.WAIT` (前提条件の順序強制)、(2) `!=` (シェル代入演算子)、(3) `?=` (条件付き代入) / `+=` (追記)。これらがPOSIX.1-2024で標準仕様に取り込まれました (`::=`はPOSIX 2012で追加済み、`:::=`は規格化されましたがGNU make 4.4.1では未実装です)。

付録B 用語集

- **.DEFAULT_GOAL**: 引数なしで `make` を実行したときの既定ゴールを明示する変数。
- **.ONESHELL**: レシピの各行を個別のシェルではなく、1つのシェルで実行するよう指定する特殊ターゲット (3.82以降)。
- **.PHONY**: 実ファイルと対応しない疑似ターゲットを宣言し、常にレシピを実行させる特殊ターゲット。
- **.POSIX**: 特殊ターゲット。宣言するとPOSIX準拠の動作になる。
- **.RECIPEPREFIX**: レシピ行の先頭文字 (デフォルトはタブ) を変更する特殊変数。
- **.SECONDEXPANSION**: 前提条件リストの二段階展開を有効にする特殊ターゲット。自動変数を前提条件で使えるようになる。
- **.WAIT**: 前提条件リスト中で、左側の前提が完了するまで右側を開始しないよう指定する特殊ターゲット (4.4新機能、POSIX.1-2024で標準化)。
- **CMake**: C/C++向けビルドシステム生成ツール。MakefileやNinjaファイルを生成する。
- **\$(eval)**: 文字列をMakefileの一部として評価 (展開) する関数。マクロと組み合わせてルールを動的生成する。
- **GNU Coding Standards**: GNUプロジェクトの開発規約。Makefileの標準ターゲットと変数名を定める。
- **Ninja**: 速度重視の低レベルビルドエンジン。Chrome・Android・LLVM等で使用。
- **order-only前提条件 (\$|)**: タイムスタンプに関わらず、存在していればよい前提条件。
- **POSIX.1-2024**: 2024年公開のIEEE規格。makeユーティリティを大幅拡張し、`.WAIT`・各代入演算子を標準化。
- **グループ化ターゲット (&:)**: 複数のターゲットが1回のレシピで生成されることを宣言する構文 (4.3以降)。
- **再帰展開 (=)**: 変数の値が参照時に毎回展開される代入方式。
- **静的なパターンルール**: 特定のターゲットのリストにだけ適用するパターンルール (`$(OBJJS): %.o: %.c`)。
- **即時展開 (:=)**: 変数の値が代入時に展開される代入方式。
- **ジョブサーバー**: 並列ビルド時に、親makeと子プロセスで並列度を共有する仕組み。4.4以降UNIXではFIFO方式。
- **ターゲット**: ルールの作成対象となる成果物の名前。
- **タブ文字**: レシピ行の先頭に必須の文字。スペースではエラーになる。
- **中間ファイル**: ビルド途中で作られ、最終ゴール作成後に自動削除されるファイル。
- **前提条件 (prerequisites)**: ターゲットより古くなったら再ビルドのトリガーとなるファイル群。
- **パターンルール**: %を使い、複数のターゲットに同じレシピを適用するルール (`%.o: %.c`)。

- 暗黙ルール: makeが組み込みで持つルール (例: `%.o: %.c`)。-r で無効化できる。
- 二次展開: `.SECONDEXPANSION` による前提条件リストの二段階展開。
- レシピ: ターゲットを作るために実行するシェルコマンド群。
- `--shuffle`: 前提条件・ゴールを並べ替えて実行し、依存宣言の漏れを検出するオプション (4.4新機能)。
- `--output-sync (-O)`: 並列ビルド時の出力を同期・整形するオプション。
- 自動変数: `$@` (ターゲット)、`$<` (最初の前提)、`$^` (全前提) など、ルール内で使える特殊変数。
- `$(MAKE)`: レシピからサブmakeを呼ぶときに使う変数。フラグとジョブサーバーを継承する。
- `$(call)`: マクロ変数を関数のように呼び出し、引数 (`$(1)~`) を渡す関数。
- `$(foreach)`: リストの各要素に対し式を展開し、結果を連結する関数。
- `$(shell)`: シェルコマンドを実行し、標準出力を返す関数。
- `$(wildcard)`: ワイルドカードに一致する実在ファイルのリストを返す関数。4.3以降ソート済み。
- `$(intcmp)`: 数値比較を行い、大小・一致に応じた値を返す関数 (4.4新機能)。
- `$(let)`: 式の中でだけ有効なローカル変数を定義する関数 (4.4新機能)。
- Make: 1976年にStuart Feldmanが開発したビルド自動化ツール (1977年説も流通)。
- Makefile Conventions: GNU Coding Standardsの一部。標準ターゲット名・変数名の慣習。
- NMAKE: Visual Studio付属のMicrosoft製make。GNU makeと構文が異なる。
- 再帰make: 親Makefileからサブディレクトリのmakeを呼び出す構成。

付録C 出典一覧

本書の記述は、以下の一次情報源を基準にしています。収集日: 2026年7月31日。

項目	出典	機関	公表日
最新版・バージョン情報	https://ftp.gnu.org/gnu/make/	GNU FSF	2023-02-26
GNU Make マニュアル (ed.0.77 / 4.4.1)	https://www.gnu.org/software/make/manual/html_node/	GNU FSF	2023-02-26
変更履歴 (NEWS)	https://raw.githubusercontent.com/mirror/make/master/NEWS	GNU	2023-02-26
開発状況	https://git.savannah.gnu.org/cgit/make.git/log/	GNU	最新 2026-04-29
Makefile Conventions	https://www.gnu.org/prep/standards/html_node/Makefile-Conventions.html	GNU/FSF	—
POSIX.1-2024 make仕様	https://pubs.opengroup.org/onlinepubs/9799919799/utilities/make.html	IEEE/The Open Group	2001-2024

項目	出典	機関	公表日
GCC Preprocessor Options (- MMD/- MP/-MF)	https://gcc.gnu.org/onlinedocs/gcc/Preprocessor-Options.html	GCC	2024
CMakeドキュメント	https://cmake.org/cmake/help/latest/	Kitware	—
Ninja公式	https://ninja-build.org/	Ninja	v1.13.1 (2025-07-10)
Bazelリリース	https://github.com/bazelbuild/bazel/releases	Bazel	9.2.0 (2026-07-13)
Task公式	https://taskfile.dev/	Go-task	v3.52.0
Just	https://github.com/casey/just	casey	v1.57.0 (2026-07-19)
NMAKE Reference	https://learn.microsoft.com/en-us/cpp/build/reference/nmake-reference	Microsoft	—
WSLインストール	https://learn.microsoft.com/en-us/windows/wsl/install	Microsoft	更新 2026-06-02
MSYS2公式	https://www.msys2.org/	MSYS2	—
Homebrew make formula	https://formulae.brew.sh/formula/make	Homebrew	—

画像・図表の出典

本書は独自のコード例と表のみで構成され、外部画像を使用していません。

免責

本書の内容は著者の理解に基づき作成されています。ツールの仕様はバージョンによって異なる場合があります。必ずご自身の環境の `make --version` と公式マニュアルで確認してください。